

SmoIVLM: Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University | [arXiv:2504.05299](https://arxiv.org/abs/2504.05299)

Compact Vision-Language Models Achieving Competitive Performance at Sub-1GB RAM

SmoIVLM-256M

256M params | 0.8 GB RAM

SmoIVLM-500M

500M params | 1.2 GB RAM

SmoIVLM-2.2B

2.2B params | 4.9 GB RAM

Large VLMs Are Powerful but Impractical for Edge Deployment

State-of-the-art VLMs require massive compute and memory, making mobile/edge deployment impossible without significant optimization.

Idefics-80B

80B params

~320 GB+

300× larger than SmolVLM-256M

Molmo-A1B-7B

7B params

27.7 GB

34× more RAM than SmolVLM-256M

InternVL2-2B

2B params

~8-10 GB

10× more RAM than SmolVLM-256M

SmolVLM-256M

256M params

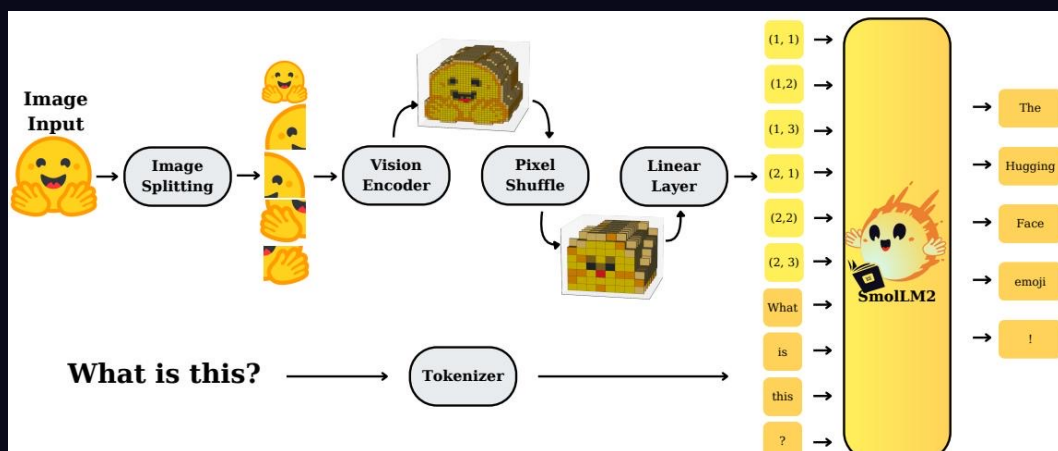
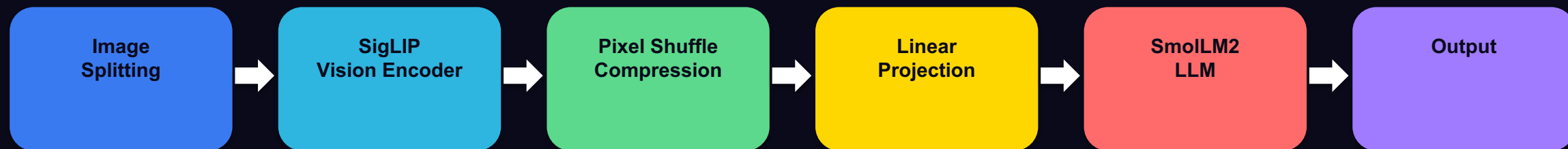
0.8 GB

Runs on smartphones & edge devices

Key Challenges:

- Massive GPU memory requirements (8-320+ GB) prevent edge deployment
- Inference latency incompatible with real-time mobile applications
- High parameter counts lead to slow throughput on limited hardware

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone



1. Image Splitting

High-resolution images split into sub-images for efficient processing

2. Vision Encoder

SigLIP-B/16 (93M) encodes each sub-image into visual features

3. Pixel Shuffle

Space-to-depth compression reduces tokens by $16\times$ at $r=4$

4. Linear Projection

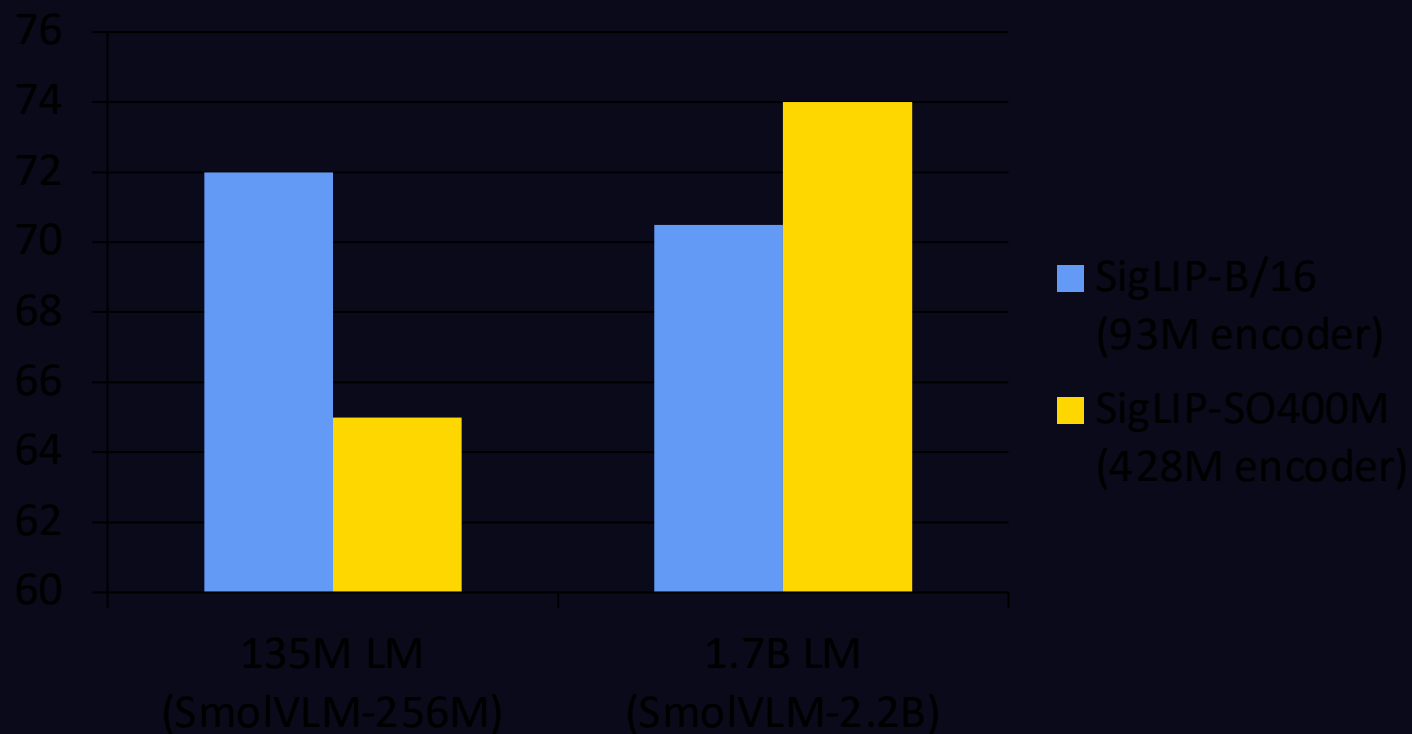
Projects vision features into SmolLM2's input embedding space

5. LLM Processing

SmolLM2 (135M–2.2B) generates text responses with fused modal tokens

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1: Compact VLMs benefit from smaller vision encoders with balanced encoder-LM parameter allocation.



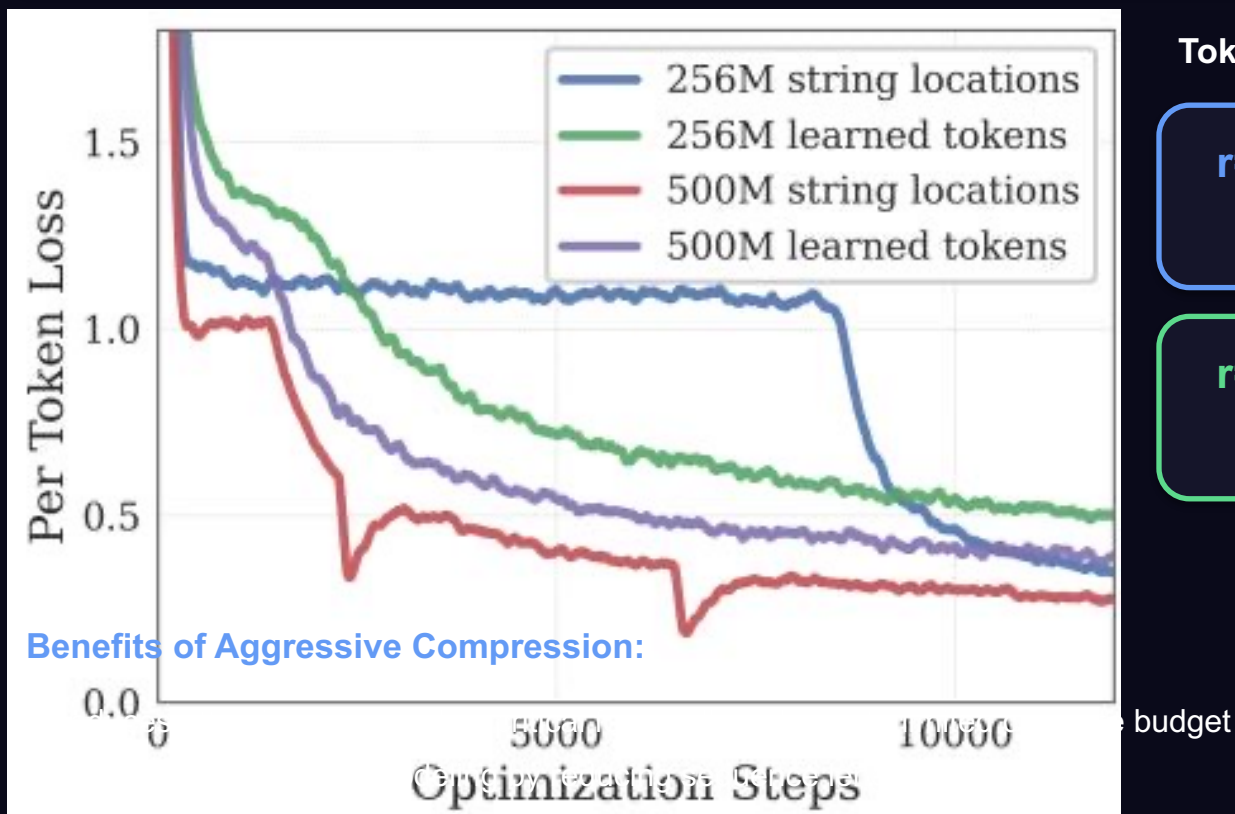
Key Insight

- 93M SigLIP-B/16 + 135M LM = best for smallest models
- At 1.7B LM scale, larger encoder justifies 10% parameter overhead
- Balanced compute allocation critical for efficiency

The 428M SigLIP-SO400M outperforms SigLIP-B/16 only when paired with a 1.7B LM. For smaller LMs, the smaller encoder's efficiency wins.

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

Finding 3: Small VLMs benefit from r=4 pixel shuffle (16× token reduction), while larger models prefer r=2.



- Enables processing of higher resolution images within same token budget
- Eases KV-cache pressure for longer generated sequences

Token Reduction by Compression Factor:

r=2

Tokens: ~4× reduction

For larger models

r=4

Tokens: 16× reduction

For small VLMs (SmoIVLM)

Extending Context from 2K to 16K Tokens Unlocks Higher Resolution

Finding 2: Performance improves significantly with extended context length from 2K to 16K tokens.

2K tokens

← 0K extension

8K tokens

← +4K extension

Technical Changes

- RoPE base: 10K → 273K (enables longer contexts)
- Fine-tuning on mixed long/short-context data
- 2.2B model: 16K context length
- Smaller variants: 8K context length

Impact on Performance

- Enables processing higher resolution images
- Better multi-image and video understanding
- Improved long document comprehension
- Critical for detailed OCR and chart tasks

Extended context allows SmolVLM to process detailed images with more visual tokens, improving benchmarks like DocVQA and ChartQA.

SmoIVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

SmoIVLM-256M

0.8 GB RAM

Avg: 44.0%

SmoIVLM-500M

1.2 GB RAM

Avg: 51.0%

SmoIVLM-2.2B

4.9 GB RAM

Avg: 59.8%

Molmo-A1B-7B

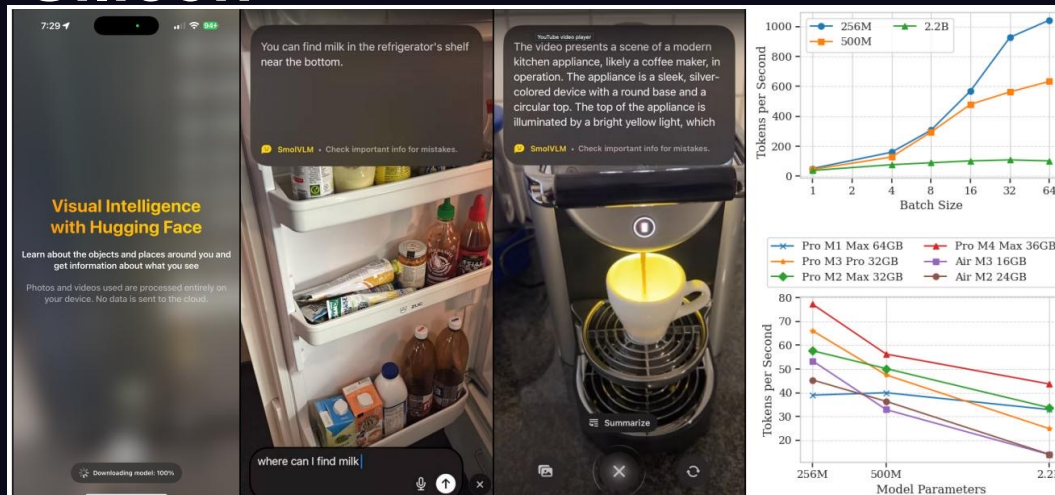
27.7 GB RAM

Avg: Best OS

Capability	Benchmark	SmoIVLM 256M	SmoIVLM 500M	SmoIVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo)
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% (Molmo)
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo)
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo)
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL)
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL)

SmoIVLM-256M surpasses the 300× larger Idefics-80B across benchmarks.

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon



Throughput on Apple Silicon:

Pro M1 Max 64GB
SmoVLM-256M

~80 tokens/sec

Pro M1 Max 64GB
Batch=64

~1000 tokens/sec

MacBook Air M3 16GB
SmoVLM-256M

~55 tokens/sec

HuggingSnap Mobile App Features:

- 100% on-device processing — no cloud dependency, complete privacy
- Processes photos and videos entirely locally on iOS devices
- Fully open-sourced: weights, datasets, code, and mobile app
- Linear scaling with batch size for smallest model
- Enables real-time multimodal applications on consumer hardware

Key Takeaways: Design Principles for Efficient Multimodal Models

1 Balance Encoder-LM Compute

A smaller 93M SigLIP-B/16 encoder with a 135M LM outperforms larger encoders in compact VLMs. Only at 1.7B+ LM scale does a larger encoder justify its overhead.

2 Extend Context Length

Increasing context from 2K to 16K via RoPE base adjustment (10K→273K) enables higher resolution processing and better long-document understanding.

3 Use Aggressive Token Compression for Small Models

Pixel shuffle at $r=4$ provides 16× token reduction, easing attention overhead and improving long-context modeling for compact VLMs.

4 Sub-1GB Inference Is Achievable

SmolVLM-256M runs at 0.8 GB RAM, enabling deployment on smartphones and edge devices without cloud dependency.

5 Open Source Drives Progress

Fully open-sourced weights, datasets, code, and HuggingSnap app promote reproducibility and accelerate on-device AI research.